

Lab 1 – Section I: Software Development Process Models

Software Engineering – Case Study: Campus Mobility App

Roman Benz

October 6, 2026

1 Software Development Life Cycle

Assignment of the activities to SDLC phases

Act.	Description	SDLC phase
C	Team determines what functionality students need	Requirements Engineering
E	Architects define components and interfaces	Design
A	Developers implement the connection search	Implementation
G	Testers check the app against the specified requirements	Testing (system test)
B	Students check whether the app meets their expectations	Testing (acceptance test)
D	Release to the university app store	Deployment
F	Fixing defects and adapting after release	Maintenance

Reasonable order

C → E → A → G → B → D → F

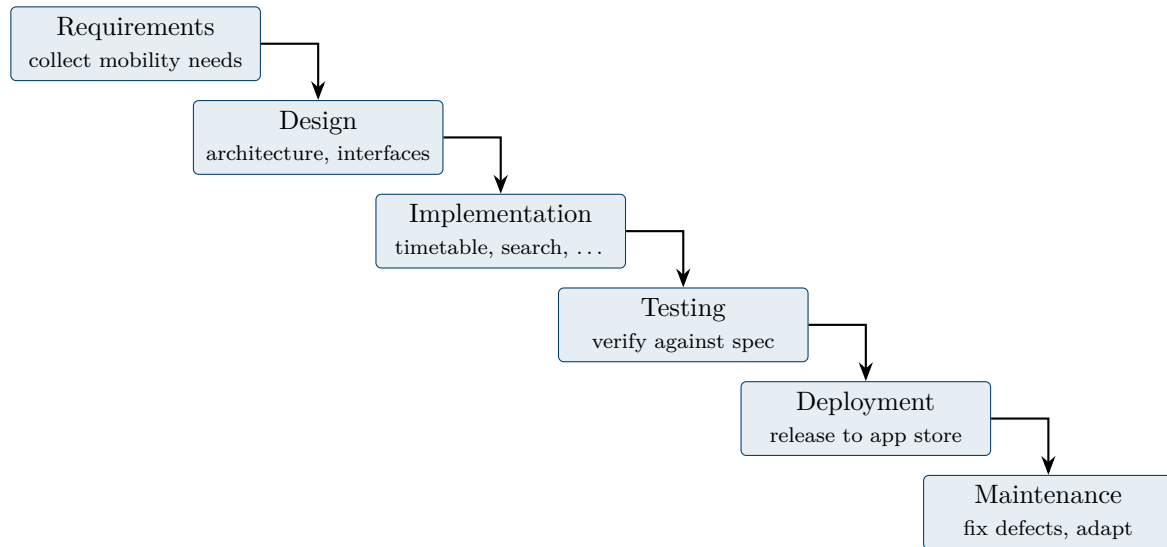
The system test (G) comes before the acceptance test (B): the team first verifies internally that the specification is fulfilled before real users validate whether the app fits their needs. Maintenance (F) is only possible after the release (D).

Typical artifacts

Activity	Typical artifact
Requirements Engineering	Requirements specification (e.g. list of functional / non-functional requirements, user stories, use case diagram)
Design	Architecture / component diagram, interface specification (e.g. API of the timetable service), data model
Implementation	Source code, executable build (app package), code documentation
Testing	Test plan and test cases, test report, defect / bug reports

2 Waterfall Model

Simplified Waterfall Model



Phase	Activities for the Campus Mobility App
Requirements	Specify shuttle overview, departure/arrival times, connection search, delay notifications, favorites
Design	Define app architecture, backend for timetable data, notification service, data model, interfaces
Implementation	Program all features of the specification
Testing	Unit, integration and system tests against the specification
Deployment	Publish the app in the university app store
Maintenance	Bug fixes, adaptations (e.g. new OS versions)

When does the Waterfall Model work well?

If the university's requirements are stable and well understood, every phase can be completed fully, documented and reviewed before the next one starts. The scope is known in advance, so cost, schedule and staffing can be planned reliably. There is little need to go back to an earlier phase, which is exactly the assumption the Waterfall Model is built on.

Problem: late request for public transport integration

In the Waterfall Model, requirements and design are considered *finished* once implementation starts. The new requirement means:

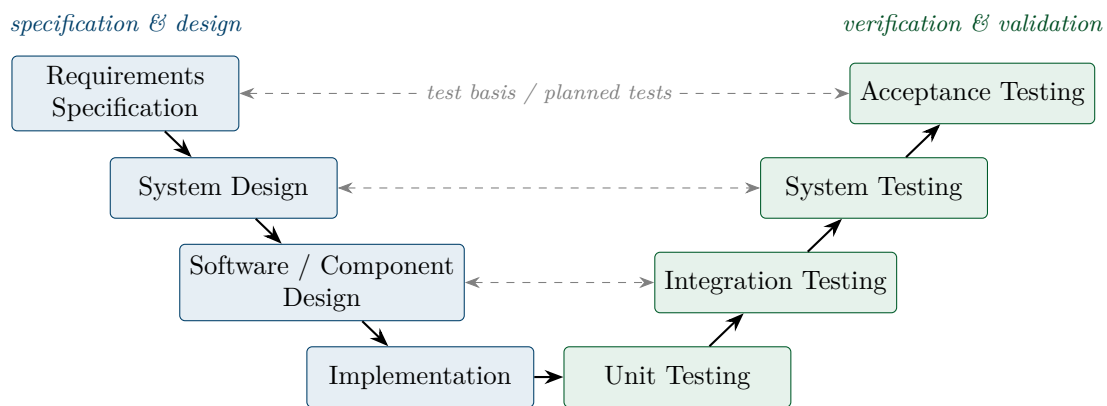
- going back to requirements *and* design – the architecture must now include an external data source (public transport API), new interfaces and a different data model;
- already implemented parts (especially the connection search) may have to be reworked, and test plans must be changed;
- the change costs much more the later it occurs, and schedule and budget no longer fit. The model has no built-in mechanism for handling changes.

Advantages and disadvantages for this project

Advantages	Disadvantages
Clear, easy-to-understand structure with defined milestones and documentation for every phase.	Changes (e.g. public transport integration, planned future features) are expensive and hard to include.
Good plannability of cost and schedule when the requirements are stable.	Students see working software only at the very end, so feedback and detected misunderstandings come very late.

3 V-Model

Simplified V-Model



Assignment of the test levels

Development activity	Test level	What is checked?
Requirements Specification	Acceptance Testing	Does the app meet the students' / university's needs?
System Design	System Testing	Does the complete system behave as specified?
Software / Component Design	Integration Testing	Do components (e.g. search, timetable service, notifications) work together via their interfaces?
Implementation	Unit Testing	Does each unit (e.g. search algorithm) work correctly?

Meaning of the connection between both sides

The horizontal connections (dashed; for implementation and unit testing the bottom of the V) show that every specification level on the left is the *test basis* for one test level on the right. The tests are *planned already* when the corresponding specification is written (e.g. acceptance tests while writing the requirements), and later they check whether the result of that level was fulfilled. This makes the relationship between development artifacts and tests explicit and traceable.

4 Iterative and Incremental Development

Three increments

Inc.	Functionality	Value for the users
1	Shuttle timetable, connection search	Core function: students see shuttles and departure/arrival times and can find a connection between university locations.
2	Delay notifications, favorites	More reliable and more comfortable: students are informed about delays and can reuse their daily routes with one tap.
3	Public transport integration, bicycle sharing, parking information	The app becomes a complete campus mobility platform covering all means of transport.

Why is the first increment already useful?

It covers the main purpose of the app: getting from one university location to another by shuttle. With timetable and connection search, students can already plan real trips. All further features (notifications, favorites, other means of transport) only make this core function more comfortable or broader. In addition, the first increment provides early feedback from real users for the following increments.

Increment vs. iteration

- **Increment:** a *usable extension* of the product – new functionality is added (“building the product piece by piece”).
- **Iteration:** a *repeated development cycle* (plan, build, test, evaluate) in which existing parts are reworked and improved (“improving the product step by step”).

Evolution of the connection search

This is **primarily iterative development**: the *same* feature (connection search) is revisited and refined in each version – first a basic search, then delay-aware, then multimodal. The feature already exists from version 1 and is improved repeatedly.

There is also an incremental aspect: version 3 adds visible new functionality (public transport). But the main focus is the step-by-step refinement of one existing feature, so “iterative” fits better.

5 Spiral Model

Why is the Spiral Model suitable?

The delay prediction is a high-risk feature: the team has no experience with machine learning prediction systems and it is unknown whether the GPS data is good enough. The Spiral Model is *risk-driven*: in each cycle, the biggest risks are identified first and reduced with prototypes or experiments, before more effort is invested. This way, the team can find out early – and cheaply – whether the feature is feasible, and can still change direction or stop.

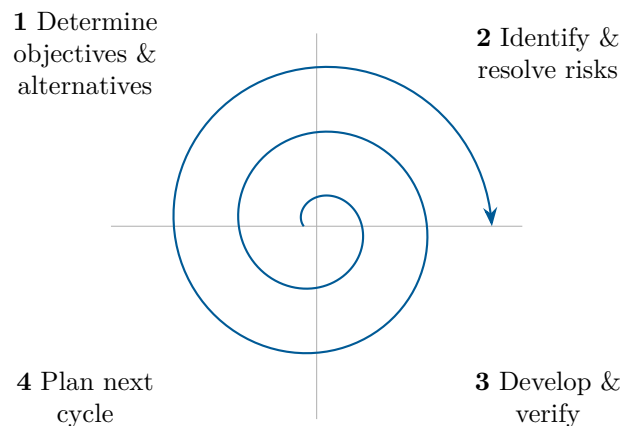
Two project risks

1. **Data quality:** GPS data may be inaccurate, incomplete, delayed or missing (e.g. signal loss, sending intervals too long).
2. **Prediction quality / know-how:** the ML model may not be accurate enough to be useful, and the team lacks experience in building and operating such a model (effort and schedule are hard to estimate).

Investigating the data-quality risk in an early spiral

1. Record GPS data of a few shuttles for some weeks together with the real departure/arrival times.
2. Analyse the data: accuracy, update frequency, gaps and outliers.
3. Build a simple prototype (e.g. a baseline model or a simple regression) and compare predicted delays with the real ones.
4. Decide based on the result: continue with the approach, improve the data source (e.g. better GPS hardware), or drop the feature.

Activities of one spiral cycle



Activity	Explanation
1. Determine objectives, alternatives, constraints	What should this cycle achieve? Which solution approaches are possible (e.g. different prediction methods)?
2. Identify and resolve risks	Evaluate the alternatives regarding their risks and reduce the biggest risks, e.g. with prototypes, simulations or analyses.
3. Develop and verify	Develop and test the (partial) product of this cycle, e.g. a prototype or a feature.
4. Plan the next cycle	Review the results with stakeholders and decide whether and how to continue (next cycle, change of direction, or stop).

6 Identify the Process Model

Scen.	Process model	Supporting characteristic
A	V-Model	“For every specification level, a corresponding test level is planned” and the traceability between artifacts and tests is documented.
B	Iterative / Incremental	“A small but usable version is released first”, then functionality is added and existing functionality is continuously improved.
C	Spiral Model	New, unknown technologies; prototypes are built “to investigate technical risks” before the final architecture is fixed.
D	Waterfall Model	Detailed and stable specification at the start; all phases are performed “largely sequentially”.

7 Final Comparison

Characteristic	Waterfall	V-Model	Iterative / Incremental	Spiral
Development mainly sequential?	Yes	Yes	No	No
Explicit relationship between development and testing?	Limited	Yes	Not central	Not central
Early delivery of partial functionality possible?	No	No	Yes	Possible
Explicit focus on risk analysis?	No	No	Not necessarily	Yes
Well suited to changing requirements?	No	Limited	Yes	Yes

Notes:

- *Waterfall* does have a test phase, but it is not explicitly linked to the individual specification levels – that is the main addition of the *V-Model*.
- In the *Spiral Model*, each cycle produces a (partial) product, so early partial delivery is possible, but its focus is risk reduction rather than delivering usable increments.
- *Iterative / incremental* development handles changing requirements well because each new iteration or increment can take the latest requirements into account.